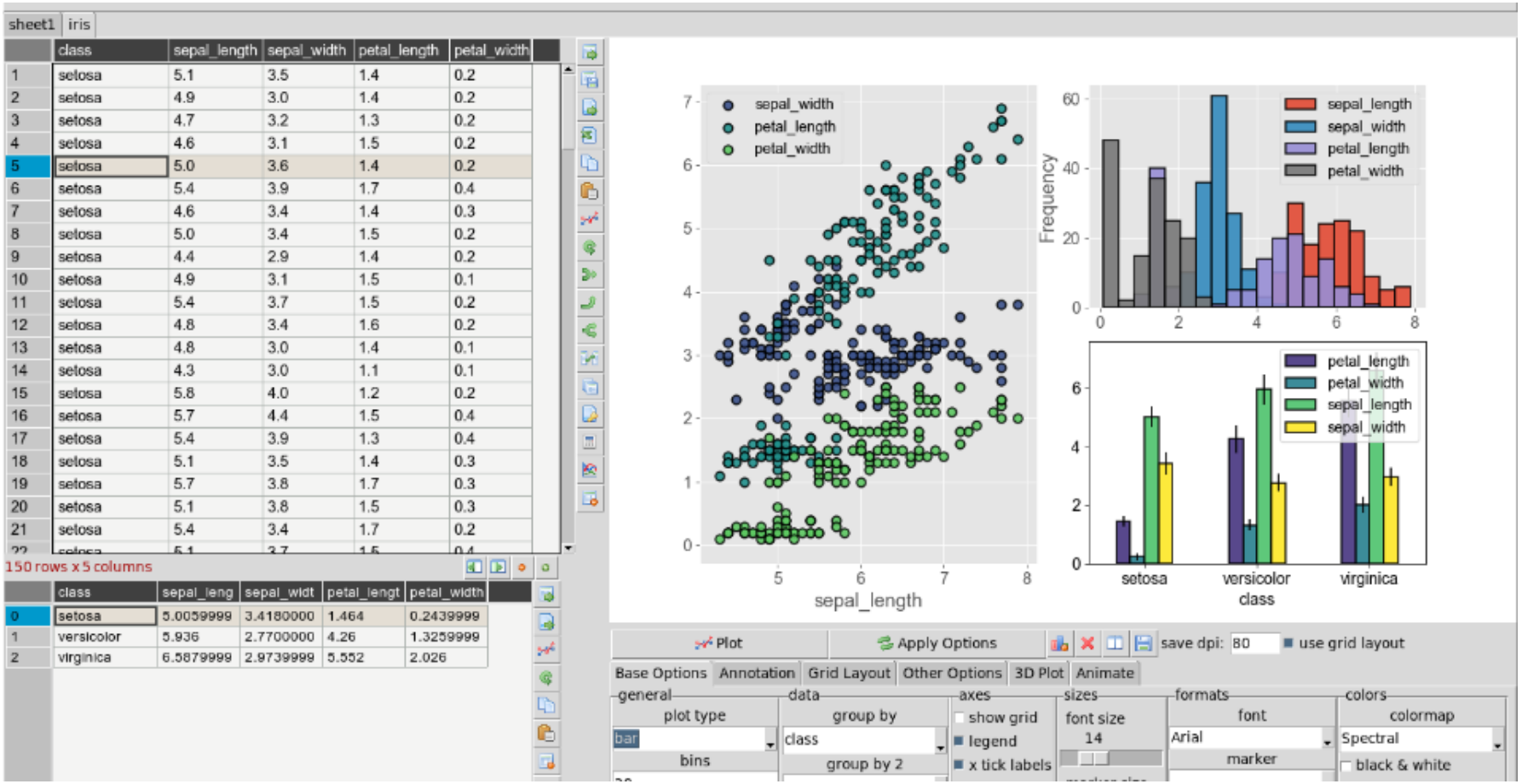
S11pandas.jpnb

Programació amb Python

Unió Europea
Fons social europeu

Pandas : Análisis de datos





Librería para el análisis de datos
Tiene estructuras de datos que permite :

- limpiar los datos en bruto
- alinear datos para su comparación,
- fusionar conjuntos de datos,
- gestión de datos perdidos,
- cálculos estadísticos

Pandas fue diseñada originalmente para gestionar datos financieros, y como alternativo al uso de hojas de cálculo (es decir, Microsoft Excel).



La librería Pandas ofrece dos de las estructuras más usadas en Data Science: la estructura **Series** y el **DataFrame**

```
import pandas as pd
```

Series

	apples
0	3
1	2
2	0
3	1

+

Series

	oranges
0	0
1	3
2	7
3	2

=

DataFrame

	apples	oranges
0	3	0
1	2	3
2	0	7
3	1	2

Agafa del servidor : s11_pandas.ipynb

Series

```
import pandas as pd

pd.Series([7,5,3], index = ["Ene", "Feb", "Mar"])
```

Un vector numérico. Puede tener índices numéricos o alfabéticos.

En el siguiente ejemplo, estamos creando una serie simplemente a partir de una lista:

```
In [17]: s = pd.Series([7, 5, 3])
         s

Out[17]: 0    7
         1    5
         2    3
         dtype: int64
```

Al no haberse especificado un índice, se asigna uno automáticamente con los valores 0, 1 y 2.

Si repetimos esta instrucción especificando un índice:

```
In [18]: s = pd.Series([7, 5, 3], index = ["Ene", "Feb", "Mar"])
         s

Out[18]: Ene    7
         Feb    5
```

DataFrame

El DataFrame es una estructura de datos tabular que se compone de columnas y filas ordenadas.

Creación de un DataFrame

Existen múltiples formas de crear un DataFrame en pandas.

DATAFRAME DESDE DICCIONARIO DE LISTAS

Cada elemento del diccionario se convierte en una **columna**, donde su clave es el nombre y sus valores se despliegan en «vertical»:

```
>>> data = {'A': [1, 2, 3], 'B': [4, 5, 6]}

>>> pd.DataFrame(data)
   A  B
0  1  4
1  2  5
2  3  6
```

DATAFRAME DESDE LISTA DE DICCIONARIOS

Cada elemento de la lista se convierte en una **fila**. Las claves de cada diccionario serán los nombres de las columnas y sus valores se despliegan en «horizontal»:

```
data = [{'A': 1, 'B': 2, 'C': 3},
        {'A': 4, 'B': 5, 'C': 6}]
pd.DataFrame(data)
   A  B  C
0  1  2  3
1  4  5  6
```

DATAFRAME DESDE LISTA DE LISTAS

Cada elemento de la lista se convierte en una **fila** y sus valores se despliegan en «horizontal». Los nombres de las columnas deben pasarse como parámetro opcional:

```
>>> data = [[1, 2], [3, 4], [5, 6]]

>>> pd.DataFrame(data, columns=['A', 'B'])
   A  B
0  1  2
1  3  4
2  5  6
```

```
import pandas as pd
ventas = pd.DataFrame({"A":[41,32,56,18],
                       "B":[17,54,6,78],
                       "C":[12,13,16,18]
print (ventas)
```

```
    A   B   C
0  41  17  12
1  32  54  13
2  56   6  16
3  18  78  18
```

```
      A   B   C
Gen  41  17  12
Feb  32  54  13
Mar  56   6  16
Abr  18  78  18
```

```
import pandas as pd
ventas = pd.DataFrame({"A":[41,32,56,18],
                       "B":[17,54,6,78],
                       "C":[12,13,16,18] },
                      index = ["Gen", "Feb", "Mar", "Abr"])
print (ventas)
```

DataFrame

El siguiente ejemplo muestra un diccionario que consta de 3 columnas o claves: Nom, Dept, DiesV (dies vacances).

```
import pandas as pd
dades = {'Nom': ['Sònia', 'Laura', 'David', 'Rosa', 'Sam'],
         'Dept' : ['PROD', 'ADMIN', 'MANT', 'ADMIN', 'PROD'],
         'DiesV' : [32, 55, 20, 43, 30]}

df = pd.DataFrame(dades)
print (df)
```

```
     Nom   Dept  DiesV
0  Sònia   PROD     32
1  Laura  ADMIN     55
2  David   MANT     20
3   Rosa  ADMIN     43
4    Sam   PROD     30
```

Filtra por la columna 'Nom'

```
1  df['Nom']
```

```
0    Sònia
1    Laura
2    David
3     Rosa
4      Sam
```

Filtra por el índice de la columna

```
1  df[1:5]  #linees de 1 a 5-1
```

	Nom	Dept	DiesV
1	Laura	ADMIN	55
2	David	MANT	20
3	Rosa	ADMIN	43
4	Sam	PROD	30

Filtra por índice y columna 'Nom'

```
print(df[1:3]['Nom'])
```

```
1    Laura
2    David
Name: Nom, dtype: object
```

Filtra por índice y una lista de columnas

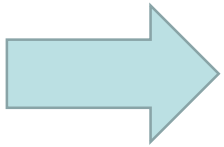
```
print(df[1:3][['Nom', 'Dept']])
```

```
     Nom   Dept
1  Laura  ADMIN
2  David   MANT
```

Condiciones :

```
df['Nom'] == 'Rosa'
```

```
0    False
1    False
2    False
3     True
4    False
Name: Nom, dtype: bool
```



```
df[df['Nom'] == 'Rosa']
```

	Nom	Dept	DiesV
3	Rosa	ADMIN	43

El método **loc** se puede usar de dos formas diferentes: seleccionar filas o columnas en base a una etiqueta o seleccionar filas o columnas en base a una condición.

```
df.loc[df['Nom'] == 'Rosa']
```

	Nom	Dept	DiesV
3	Rosa	ADMIN	43

```
print(df.loc[1])
```

```
Nom       Laura
Dept      ADMIN
DiesV        55
Name: 1, dtype: object
```

El método **iloc** se utiliza en los **DataFrames** para seleccionar los elementos únicamente en base a su ubicación física, que empieza en cero.
Su sintaxis es `data.iloc[<filas>, <columnas>]`

```
df.iloc[0:5]          # Primeras cinco filas
df.iloc[:, 0:5]       # Primeras cinco columnas
df.iloc[[0,2,1]]      # Primera, tercera y segunda filas
df.iloc[:, [0,2,1]] # Primera, tercera y segunda columnas
```

Es importante tener en cuenta que **iloc**

- devuelve una **Serie Pandas** cuando se selecciona una fila
- y un **DataFrame** cuando se selecciona varias.

En el caso que sea necesario seleccionar un **DataFrame** con una única columna es necesario pasar una lista con la columna, no un escalar.

Pandas comandos básicos

https://www.youtube.com/watch?v=PvNKKrPE0AI

`df.shape`	Muestra la dimensión del dataframe
`df.columns`	Muestra el nombre de las columnas
`df.index`	Muestra rango de los índices

`df.head(n)`	Muestra las primeras líneas (def 5)
`df.tail(n)`	Muestra las últimas líneas (def 5)
`df.describe()`	Muestra estadísticas básicas
`df.sort_values(by='tmin')`	Ordena por una columna
`df.T`	Transponer la matriz (cambiar filas x columnas)

```
df[df['Dept'] == "ADMIN"]
```

Filtrar por un valor de la columna Dept.

	Nom	Dept	DiesV
1	Laura	ADMIN	55
3	Rosa	ADMIN	43

Ordenar por la columna DiesV descendente

```
df.sort_values(by=['DiesV'], ascending=False)
```

	Nom	Dept	DiesV
1	Laura	ADMIN	55
3	Rosa	ADMIN	43
0	Sònia	PROD	32
4	Sam	PROD	30
2	David	MANT	20

```
df.sort_values(by=[ 'Dept', 'DiesV',], ascending=False)
```

	Nom	Dept	DiesV
0	Sònia	PROD	32
4	Sam	PROD	30
2	David	MANT	20
1	Laura	ADMIN	55
3	Rosa	ADMIN	43

Añadir una columna

```
df['PreuDia'] = [60, 80, 90, 100, 85]
print (df)
```

	Nom	Dept	DiesV	PreuDia
0	Sònia	PROD	32	60
1	Laura	ADMIN	55	80
2	David	MANT	20	90
3	Rosa	ADMIN	43	100
4	Sam	PROD	30	85

Añadir una fila usando el índice

```
df.loc[5] = ['Manel', 'MANT', 5, 90]
print (df)
```

	Nom	Dept	DiesV	PreuDia
0	Sònia	PROD	32	60
1	Laura	ADMIN	55	80
2	David	MANT	20	90
3	Rosa	ADMIN	43	100
4	Sam	PROD	30	85
5	Manel	MANT	5	90

Añadir una fila usando append

```
df=df.append({'Nom':'Sara' , 'Dept':'VENDES', 'DiesV':8, 'PreuDia':44} , ignore_index=True)
print(df)
```

	Nom	Dept	DiesV	PreuDia
0	Sònia	PROD	32	60
1	Laura	ADMIN	55	80
2	David	MANT	20	90
3	Rosa	ADMIN	43	100
4	Sam	PROD	30	85
5	Manel	MANT	5	90
6	Sara	VENDES	8	44

Eliminar filas

https://www.freecodecamp.org/news/drop-list-of-rows-from-pandas-dataframe/

Índicadores estadísticos del DataFrame

```
1 df.describe()
```

	DiesV	PreuDia
count	7.000000	7.000000
mean	27.571429	78.428571
std	18.100250	19.594703
min	5.000000	44.000000
25%	14.000000	70.000000
50%	30.000000	85.000000
75%	37.500000	90.000000
max	55.000000	100.000000

Práctica P01-1

```
dades = {'Nom': ['Sònia', 'Laura', 'David', 'Rosa', 'Sam'],
         'Dept' : ['PROD', 'ADMIN', 'MANT', 'ADMIN', 'PROD'],
         'DiesV' : [32, 55, 20, 43, 30]}

df = pd.DataFrame(dades)
print (df)
```

Ejercicio 1: Añade al dataframe df : 1 persona para cada departamento con 5, 7 y 9 días de vacaciones.
Muestra la lista de empleados ordenados por Dept, DiesV.

.

Ejercicio 2: Dado este diccionario crea un DataFrame de 2 columnas, nombre y nota :

```
notas={'Juan':9.0,'María':6.5,'Pablo':4.0,'Carmen':8.5,'Luis':5.0}
```

Calcula la nota mínima, la máxima, la media y la desviación típica.

Ejercicio 3: añade 3 personas al diccionario notas.

a- Ordena las notas de menor a mayor
b- Muestra solamente la nota de Juan.
c- Haz una selección de las personas suspendidas

Práctica 1-2

Ejercicio 1: Crea un DataFrame adaptando estos datos

```
ip Provincia, Población
('01','Andalucía',  8500187),
('02','Aragón', 1326315),
('03','Asturias', 1004686),
('04','Balears', 1176659),
('05','Canarias' , 2177701),
('06','Cantabria' , 585402),
('07','Castilla', 2372640)]
```

Ejercicio 2: Crea un DataFrame adaptando estos datos. Edad Media de la población.

	2022	2021	2020	2019	2018	2017	2016
POBLACIÓN	44,08	43,81	43,55	43,34	43,13	42,91	42,67
Españoles	45,00	44,74	44,50	44,21	43,94	43,68	43,45
Extranjeros	37,08	36,63	36,23	36,13	36,01	35,79	35,56

SOLUCIONS P01-2

Ejercicio 1.1

```
datos = [
('01','Andalucía',  8500187),
('02','Aragón', 1326315),
('03','Asturias', 1004686),
('04','Balears', 1176659),
('05','Canarias' , 2177701),
('06','Cantabria' , 585402),
('07','Castilla', 2372640)]

print (pd.DataFrame(datos, columns=('ip', 'Provincia','Población')))
```

Solución P01-2 Ejercicio 1.2

```
datos2 = {
'P.General'   :      (44.08,43.81,43.55,43.34,43.13,42.91,42.67),
'Españoles'   :      (45.00,44.74,44.50,44.21,43.94,43.68,43.45),
'Extranjeros' :      (37.08,36.63,36.23,36.13,36.01,35.79,35.56)
}

print (pd.DataFrame(datos2, index = (2022,2021,2020,2019,2018,2017,2016)))
```

Leer un fichero csv en pandas

Prueba desde Jupyter Notebook

```
import pandas as pd

df = pd.read_csv(r".\\dat\\obser2021.csv", sep=";")
df
```

Ya tenemos un dataframe de tipo pandas. Vamos a hacer consultas

	anyy	mes	dia	obser	ventkh	component	plujamm	tmax	tmin	tmit
0	1971	2	18	STACOL	38	NO	9	11.29	8.33	1.48
1	1968	1	18	MASNOU	48	S	6	38.41	17.46	10.47
2	2003	4	9	LLEIDA	5	S	7	20.21	-4.42	12.32
3	1961	10	21	HOSPI	200	S	3	24.31	-3.57	13.94
4	1968	2	17	BCN	144	N	1	24.75	-8.13	16.44
5	1992	10	8	BERGA	0	N	5	16.79	5.88	5.46

Mostrar el tipo de datos de cada columna

df.dtypes

```
df.head()
```

head() muestra las primeras lineas del fichero

Out[5]:

	anyy	mes	dia	obser	ventkh	component	plujamm	tmax	tmin	tmit
0	1971	2	18	STACOL	38	NO	9	11.29	8.33	1.48
1	1968	1	18	MASNOU	48	S	6	38.41	17.46	10.47
2	2003	4	9	LLEIDA	5	S	7	20.21	-4.42	12.32
3	1961	10	21	HOSPI	200	S	3	24.31	-3.57	13.94
4	1968	2	17	BCN	144	N	1	24.75	-8.13	16.44

```
df.tail()
```

ut[6]:

	anyy	mes	dia	obser	ventkh	component	plujamm	tmax	tmin	tmit
4994	2008	3	12	LLEIDA	114	N	6	7.54	-9.49	8.52
4995	1973	5	14	BCN	124	E	1	33.52	10.77	11.38
4996	1973	12	11	HOSPI	41	SO	10	20.49	2.91	8.79
4997	2014	5	24	HOSPI	143	SE	9	9.22	3.46	2.88
4998	1992	1	25	STACOL	29	NO	1	42.36	-5.30	23.83

```
In [7]: df.describe()
```

Out[7]:

	anyy	mes	dia	ventkh	plujamm	tmax	tmin	tmit
count	4999.000000	4999.000000	4999.000000	4999.000000	4999.000000	4999.000000	4999.000000	4999.000000
mean	1984.137828	6.563313	15.300660	100.344669	4.981396	25.296983	4.943527	10.176759
std	19.975886	3.471767	8.678153	57.680564	3.126686	12.398262	8.579557	6.380774
min	1950.000000	1.000000	1.000000	0.000000	0.000000	-9.920000	-10.000000	0.000000
25%	1967.000000	4.000000	8.000000	50.000000	2.000000	16.430000	-2.500000	4.905000
50%	1984.000000	7.000000	15.000000	101.000000	5.000000	26.180000	4.870000	9.570000
75%	2002.000000	10.000000	23.000000	149.000000	8.000000	35.805000	12.305000	14.690000
max	2018.000000	12.000000	30.000000	200.000000	10.000000	45.000000	20.000000	27.270000

Calcula los parámetros básicos de la muestra :

recuento, media, std = desviación standard, mínimo, percentiles, máximo.

df.std(axis = 0, skipna = True)

In [8]:
```
df.T
```

Matriz traspuesta

Out[8]:

	0	1	2	3	4	5	6	7
anyy	1971	1968	2003	1961	1968	1992	1986	2013
mes	2	1	4	10	2	10	4	7
dia	18	18	9	21	17	8	22	16
obser	STACOL	MASNOU	LLEIDA	HOSPI	BCN	BERGA	TIBIDABO	BERGA
ventkh	38	48	5	200	144	0	176	66
component	NO	S	S	S	N	N	NE	SO
plujamm	9	6	7	3	1	5	10	5
tmax	11.29	38.41	20.21	24.31	24.75	16.79	31.58	21.73
tmin	8.33	17.46	-4.42	-3.57	-8.13	5.88	7.76	18.28
tmit	1.48	10.47	12.32	13.94	16.44	5.46	11.91	1.72

10 rows × 4999 columns

Seleccionar varias columnas

```
1 df[['dia', 'ventkh', 'component']]
```

	dia	ventkh	component
0	9	191	NO
1	24	138	N
2	6	192	NE
3	12	36	NE
4	10	114	S

Lista y tipos de columnas del dataframe `df.columns.tolist()`

Ver columnas con get `df.get(["anyy", "mes", "tmax"])`

Ver la columna año del dataframe `df['anyy']`

La temperatura máxima, máxima del dataframe `df['tmax'].max()`

```
df['obser'].value_counts()
```

```
TARRACO     532
MASNOU      523
TIBIDABO    517
BCN         510
STACOL      502
VIC         500
LLEIDA      490
GIRONA      483
BERGA       479
HOSPI       463
```

Selecciona y cuenta por columna

Filtrar por observatorio BCN

```
df2 = df[df['obser'] == "BCN"]
print (df2)
```

	anyy	mes	dia	obser	ventkh	component	plujamm	tmax	tmin	tmitjana
1	2011	10	24	BCN	138	N	6	39.77	16.68	11.55
5	1974	7	2	BCN	151	E	4	32.82	19.00	6.91
10	1954	7	8	BCN	15	SE	0	22.21	10.92	5.65
11	2006	7	15	BCN	171	S	8	14.92	4.29	5.31
35	1967	9	26	BCN	141	N	10	14.23	-8.29	11.26
37	1994	4	11	BCN	18	N	8	18.04	13.07	2.48

Concatenar comandos sobre dataframes suele generar nuevos dataframes, si se trata se selecciones de elementos.

```
resultado = df[['anyy','mes','tmax']][df['obser'] == "BCN"]
print (resultado)
```

	anyy	mes	tmax
1	2011	10	39.77
5	1974	7	32.82
10	1954	7	22.21
11	2006	7	14.92
35	1967	9	14.23
37	1994	4	18.04
42	2010	8	15.52

Si estamos realizando cálculos, el resultado puede ser un valor.

```
resultado = df['anyy'][df['obser'] == "BCN"].max()
print (resultado)
```

df.sort_values(by='tmax')

```
df.sort_values(by='plujamm', ascending=False)
```

	anyy	mes	dia	obser	ventkh	component	plujamm	tmax	tmin	tmit
1251	1995	1	30	MASNOU	136	SO	10	29.81	3.67	13.07
424	2002	4	30	GIRONA	198	SE	10	30.83	2.72	14.05
420	2001	5	6	LLEIDA	130	S	10	34.02	17.60	8.21

```
 df[:3]                  # Muestra las filas especificadas
 df[['anyy','mes']]      # Muestra las columnas especificadas
 df[['anyy','mes']][:3] # cols y lins especificadas
 df[df['tmin'] > 15]     # muestra registros con tmin > 15
```

```
df['tmin'].mean()           # obtén la media
df['tmin'].median()         # obtén la mediana

# pasa la columna tmin a negativo
df['tmin'].apply (lambda x: -x)

# rellena la columna tmin con los valores al cuadrado
df['tmin'].apply (lambda x: x**2)
```

df.groupby(by='tmax') Agrupar por columnas

Agrupa los valores por una o varias columnas y realiza una operación con las columnas restantes.

```
# Agrupa por año y suma
df.groupby(['anyy']).sum()
```

anyy	mes	dia	ventkh	plujamm	tmax	tmin	tmitjana
1950	450	1018	7007	298	1663.57	367.41	648.08
1951	509	1219	6901	393	1929.10	427.96	750.55
1952	455	1175	7095	370	1972.47	342.88	814.75

```
#Agrupa por año y calcula
#la media de la columna tmin
df.groupby('anyy')['tmin'].mean()
```

```
df.groupby('anyy')['tmin'].mean()
anyy
1950    5.483731
1951    5.631053
1952    4.696986
1953    6.938941
1954    3.444416
```

```
# Agrupar por dos columnas y obtiene los valores medios
df.groupby(['anyy', 'mes']).mean()
```

anyy	mes	dia	ventkh	plujamm	tmax	tmin	tmitjana
1950	**1**	13.500000	132.500000	4.000000	27.030000	1.230000	12.902500
	2	19.857143	82.714286	4.857143	20.888571	7.942857	6.474286
	3	20.833333	88.666667	6.166667	32.586667	8.680000	11.953333
	4	10.000000	129.000000	5.800000	29.430000	7.460000	10.984000
	5	21.000000	93.600000	5.000000	16.052000	2.794000	6.628000

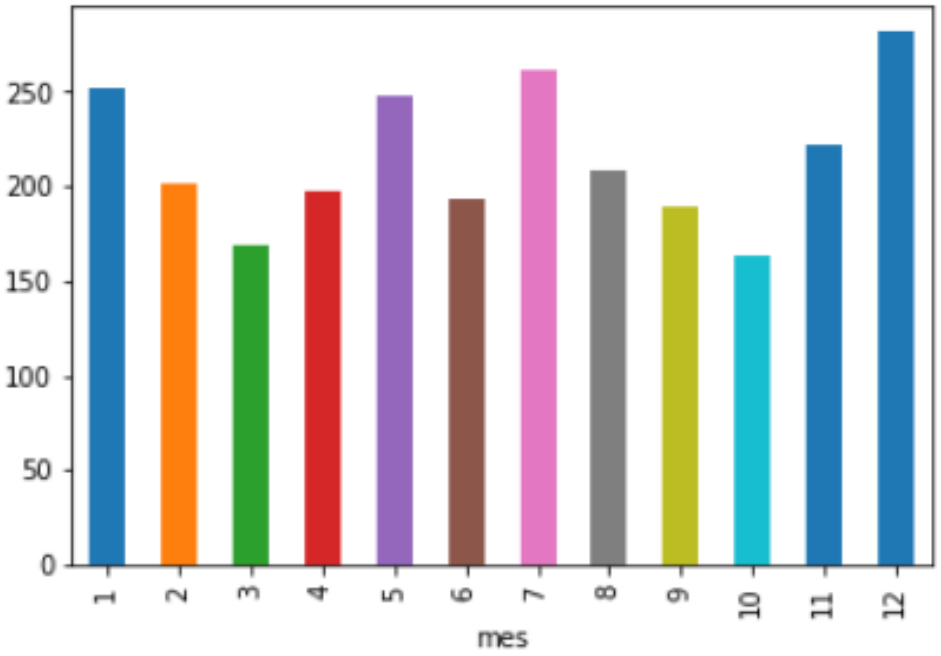
Recuerda el resultado de una agrupación NO ES un DataFrame !!!

```
import matplotlib.pyplot as plt

plot_data = df[df['obser'] == 'BCN']
plot_data = plot_data.groupby('mes')['plujamm'].sum()
plot_data.plot(kind='bar')
```

Agrupar los datos del observatorio de Barcelona y agruparlo por mes, sumando el volumen de lluvia

Realizar un gráfico de barras del DataFrame.



https://pandas.pydata.org/docs/user_guide/10min.html

Práctica P02

Ejercicio 1 : Carga en tu cuaderno python el siguiente fichero : obser2021.csv

Ejercicio 2: Que hace esta programación ?

```
df2 = df[df['anyy'] > 2016]
df2 = df2[['anyy', 'mes', 'plujamm']]
df2.groupby(['anyy']).sum()
```

Ejercicio 3: Analiza los datos de los 5 últimos años con datos.

a. Localiza el observatorio con la temperatura máxima
b. Localiza el observatorio con la temperatura mínima en Febrero
c. Observatorio, día mes y año con el viento más fuerte

Ejercicio 4: Queremos saber las temperaturas máximas de cada mes en los distintos observatorios y en los distintos años.

- Cómo presentarías los datos ?
- Que criterio utilizarías para decidir cual es el año más frío ?

Vamos a estudiar los valores nulos

```
import numpy as np
#Creamos 2 valores nulos

#df['tmax'][:2] = np.nan

#y vemos como queda el dataframe
#df.isna().sum()

#df.isna().sum().sum()

#df['tmax'].fillna(2)

#df['tmax'] = df['tmax'].fillna(2)
#df.isna().sum()
```

Best Python libraries for Machine Learning

https://www.geeksforgeeks.org/best-python-libraries-for-machine-learning/

Pandas

https://www.youtube.com/watch?v=CmorAWRsCAw

Pandas en 14 minutos

https://www.youtube.com/watch?v=PvNKKrPE0AI

Práctica P03. Con Alf

Tutorial

https://aprendeconalf.es/docencia/python/manual/pandas/

Ejercicios

https://aprendeconalf.es/docencia/python/ejercicios/pandas

Cuidado con el copiar y pegar de esta web. Si te dan problemas los espacios dan problemas, pégalo a otro editor (notepad o notepad++)

Ejercicio

Observa la creación de estos datos.

- Selecciona la isla más poblada.
- Ordena las islas por provincia.

	Island	Population	Area	Province
0	Gran Canaria	855521	1560.10	LPGC
1	Tenerife	928604	2034.38	SCTF
2	La Palma	83458	708.32	SCTF
3	Lanzarote	155812	845.94	LPGC
4	La Gomera	21678	369.76	SCTF
5	El Hierro	11147	278.71	SCTF
6	Fuerteventura	119732	1659.00	LPGC

```
import pandas as pd

islands = ['Gran Canaria', 'Tenerife', 'La Palma','Lanzarote', 'La Gomera',
    'El Hierro','Fuerteventura']

populations = [855521, 928604, 83458, 155812, 21678, 11147, 119732]
areas = [1560.1, 2034.38, 708.32, 845.94, 369.76, 278.71, 1659]
provinces = ['LPGC', 'SCTF', 'SCTF', 'LPGC', 'SCTF', 'SCTF', 'LPGC']

data = dict(Island=islands, Population=populations, Area=areas,
Province=provinces)

canarias = pd.DataFrame(data)

#canarias.set_index('Island')
```

Ejemplo: Agrupar con agregaciones .agg

```
dm = pd.DataFrame( { "A": [1, 1, 2, 2],
                     "B": [1, 2, 3, 4],
                     "C": [0.3628, 0.2278, 1.2677, -0.5628],
                   })
```

	A	B	C
0	1	1	0.3628
1	1	2	0.2278
2	2	3	1.2677
3	2	4	-0.5628

```
>>> dm.groupby('A').agg('min')
```

A	B	C
1	1	0.227877
2	3	-0.562860

Multiple aggregations

```
dm.groupby('A').agg(['min', 'max'])
```

	B		C	
	min	max	min	max
A				
1	1	2	0.227877	0.362838
2	3	4	-0.562860	1.267767

Select a column for aggregation

Different aggregations per column

```
dm.groupby('A').agg({'B': ['min', 'max'], 'C': 'sum'})
```

	B		C
	min	max	sum
A			
1	1	2	0.590715
2	3	4	0.704907

Solucions




```
# SOLUCIONS P01-1
# Ejercicio 1 : Vacances

dades = {'Nom': ['Sònia', 'Laura', 'David', 'Rosa', 'Sam', 'Pere','Pau','Pep'],
        'Dept' : ['PROD', 'ADMIN', 'MANT', 'ADMIN', 'PROD', 'PROD', 'MANT', 'ADMIN'],
        'DiesV' : [32, 55, 20, 43, 30, 5, 7, 9]}

df = pd.DataFrame(dades)
print (df.sort_values(by=['Dept','DiesV']))
```

```
# Ejercicio 2: Dado este diccionario crea un DataFrame

notas={'Juan':9.0,'María':6.5,'Pablo':4.0,'Carmen':8.5,'Luis':5.0}
dades = {'nom':notas.keys(), 'nota':notas.values()}

df = pd.DataFrame(dades)
print(df)
```

```
# SOLUCIONS P02
# Ejercicio 1 : Carga en tu cuaderno python el siguiente
fichero : obser2021.csv

import numpy as np
import pandas as pd

df = pd.read_csv("./dat/obser2021.csv", sep=";")
```

Ejercicio 2: Que hace esta programación ?

```
df2 = df[df['anyy'] > 2016]
df2 = df2[['anyy', 'mes', 'plujamm']]
df2.groupby(['anyy']).sum()
```

anyy	mes	plujamm
2017	519	358
2018	390	344

Ejercicio 3: a) Analiza los datos de los 5 últimos años con datos.

```
df3 = df.groupby(['anyy']).mean()
df3.sort_values(by=['anyy'], ascending=False)[0:5]
```

anyy	mes	dia	ventkh	plujamm	tmax	tmin	tmitjana
2018	6.393443	14.065574	105.081967	5.639344	25.508197	5.065574	10.220820
2017	6.740260	15.116883	103.753247	4.649351	23.831169	4.403766	9.714286
2016	6.352941	15.220588	100.485294	5.661765	24.116765	5.325441	9.395441
2015	6.056338	13.408451	97.000000	4.478873	24.252958	5.954930	9.149014
2014	6.454545	15.688312	97.285714	4.454545	25.570390	5.092208	10.238701

```
# Ejercicio 3: b)
# Localiza el observatorio con la temperatura máxima

df5a = df[df['anyy'] > 2013]
max = df5a ['tmax'].max()

df5a.loc[df5a['tmax'] == max]
```

	anyy	mes	dia	obser	ventkh	component	plujamm	tmax	tmin	tmitjana
2576	2014	11	27	TARRACO	8	N	0	44.99	17.59	13.7

```
# Ejercicio 3: c)
# Localiza el observatorio con la temperatura mínima en Febrero

df5aFeb = df5a[df5a['mes'] == 2]
min = df5aFeb ['tmin'].min()

df5aFeb.loc[df5aFeb['tmin'] == min]
```

	anyy	mes	dia	obser	ventkh	component	plujamm	tmax	tmin	tmitjana
2735	2017	2	26	TARRACO	89	SO	4	14.97	-9.23	12.1

```
# Ejercicio 3: d)
# Observatorio, día mes y año con el viento más fuerte

vent_max = df5a ['ventkh'].max()
df5a.loc[df5a['ventkh'] == vent_max].iloc [:,[3,2,1,0,4]]
```

	obser	dia	mes	anyy	ventkh
1266	LLEIDA	5	5	2014	200
2406	BERGA	26	8	2014	200

```
# Ejercicio 4: Queremos saber las temperaturas máximas de cada mes en los distintos observatorios y en los distintos años.
# Cómo presentarías los datos ?
# Que criterio utilizarías para decidir cual es el año más frío ?

df4a = df.groupby(['obser','anyy', 'mes']).agg({'tmax': ['max']})
df4a.tail()
```

			tmax
			max
obser	anyy	mes	
VIC	2017	4	16.37
		10	29.60
	2018	1	25.73
		2	42.04
		4	25.47

```
df4b = df.groupby(['anyy']).mean()

df4b.sort_values(by=['tmin'])[0:1]
```

anyy	mes	dia	ventkh	plujamm	tmax	tmin	tmitjana
1957	7.0	15.493333	101.68	4.4	20.940533	1.889333	9.525867